



DR B. DIOP

babacar.diop@ussein.edu.sn

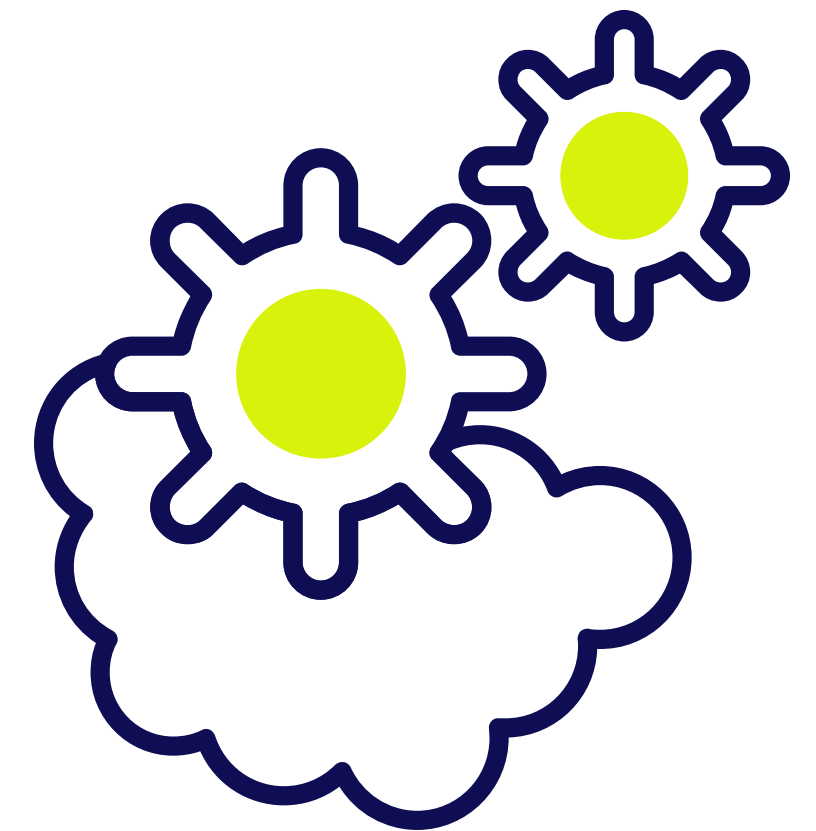
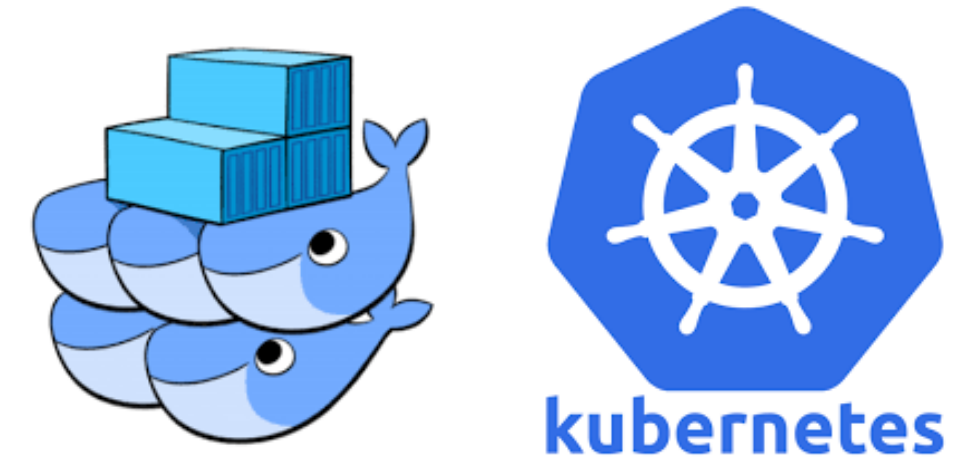
Conteneurisation & orchestration

Kubernetes Orchestration

Deployment

INFO – AGROTIC

Last update: dec. 2025



Principe du “Deployment”

La spécification de déploiement est un objet YAML déclaratif dans lequel on décrit l'**état souhaité** d'une application sans état.

Exemple de fichier YAML

```
yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mon-app
  labels:
    app: mon-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: mon-app
  template:
    metadata:
      labels:
        app: mon-app
    spec:
      containers:
      - name: mon-app
        image: mon-image:tag
        ports:
        - containerPort: 8080
```

Principe du “Deployment”

Le fichier **.yaml** est transmis à Kubernetes, où le contrôleur de déploiement l'implémente et le gère.

Le contrôleur est hautement disponible et fonctionne comme une boucle en arrière-plan qui réconcilie l'**état observé** avec l'**état souhaité**.

```
yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mon-app
  labels:
    app: mon-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: mon-app
  template:
    metadata:
      labels:
        app: mon-app
    spec:
      containers:
        - name: mon-app
          image: mon-image:tag
          ports:
            - containerPort: 8080
```

Principe du “Deployment”

Les objets de déploiement, ainsi que toutes leurs fonctionnalités et attributs, sont définis dans le sous-groupe API **apps/v1** workloads.

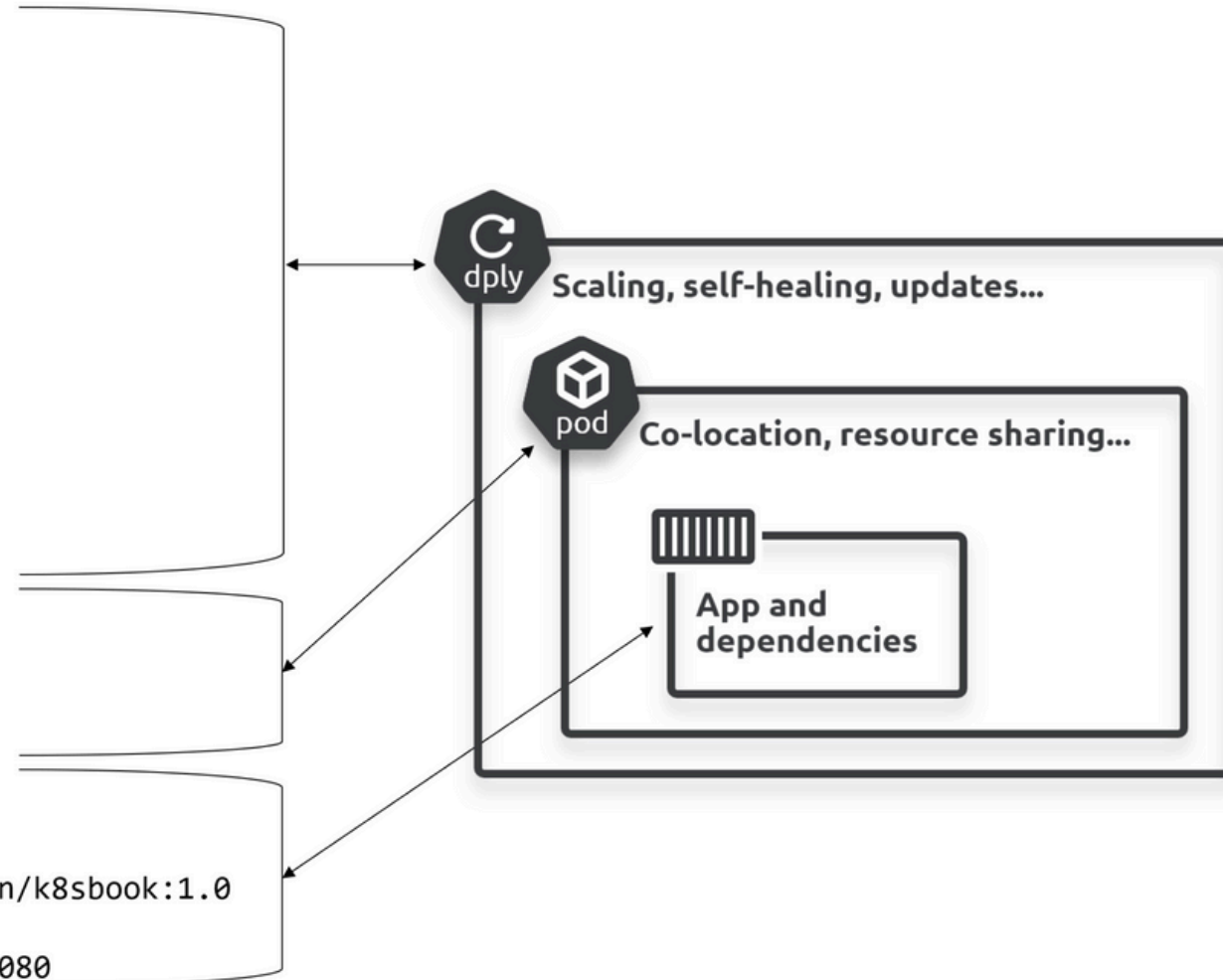
```
yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mon-app
  labels:
    app: mon-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: mon-app
  template:
    metadata:
      labels:
        app: mon-app
    spec:
      containers:
        - name: mon-app
          image: mon-image:tag
          ports:
            - containerPort: 8080
```

“Deployment” : meilleur que les Pods

- En pratique, on peut déployer une application sans état, sous forme de conteneur (**docker** par exemple), puis on la définit dans un modèle de **pod**, où l'on peut l'exécuter sur **Kubernetes**.
- *Cependant, les **Pods statiques** de ce type ne sont pas auto-réparables, ils ne sont pas évolutifs et ne permettent pas de mises à jour et de restaurations faciles.*
- **Solution**: Pour ces raisons, il faut intégrer un **Pod** presque toujours dans un objet **Deployment**.

“Deployment” comme solution aux Pods

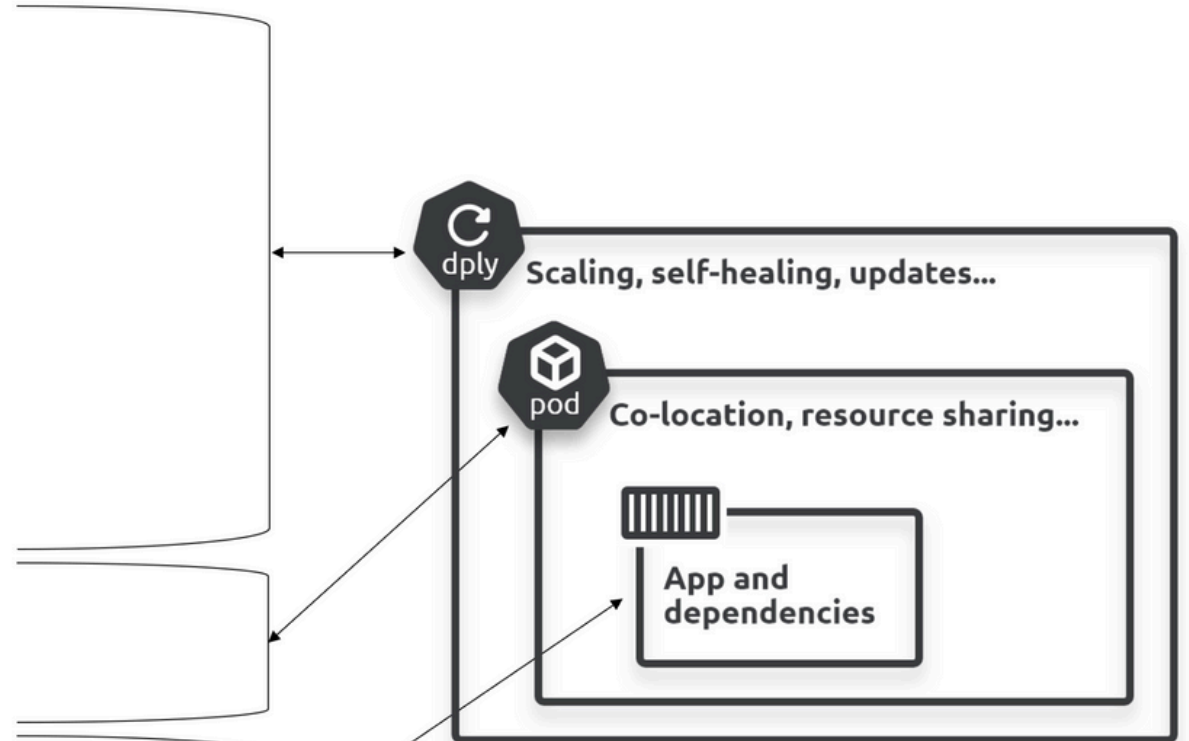
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 10
  selector:
    matchLabels:
      app: hello-world
  minReadySeconds: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-pod
          image: nigelpoulton/k8sbook:1.0
          ports:
            - containerPort: 8080
```



“Deployment” comme solution aux Pods

1. Le conteneur héberge l'application. L'image de l'application est nigelpoulton/k8sbook:1.0 sur le port 8080

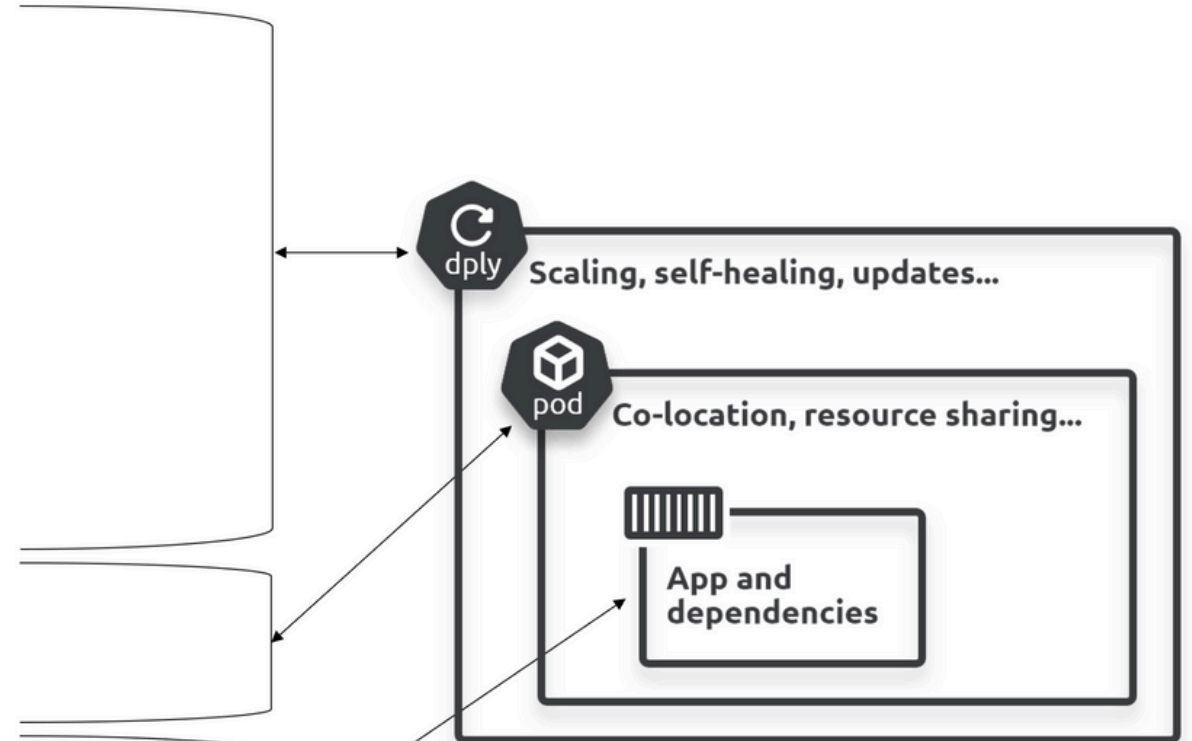
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 10
  selector:
    matchLabels:
      app: hello-world
  minReadySeconds: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-pod
          image: nigelpoulton/k8sbook:1.0
          ports:
            - containerPort: 8080
```



“Deployment” comme solution aux Pods

2. Le Pod enrichit le conteneur avec des étiquettes, des annotations et d'autres métadonnées utiles pour Kubernetes

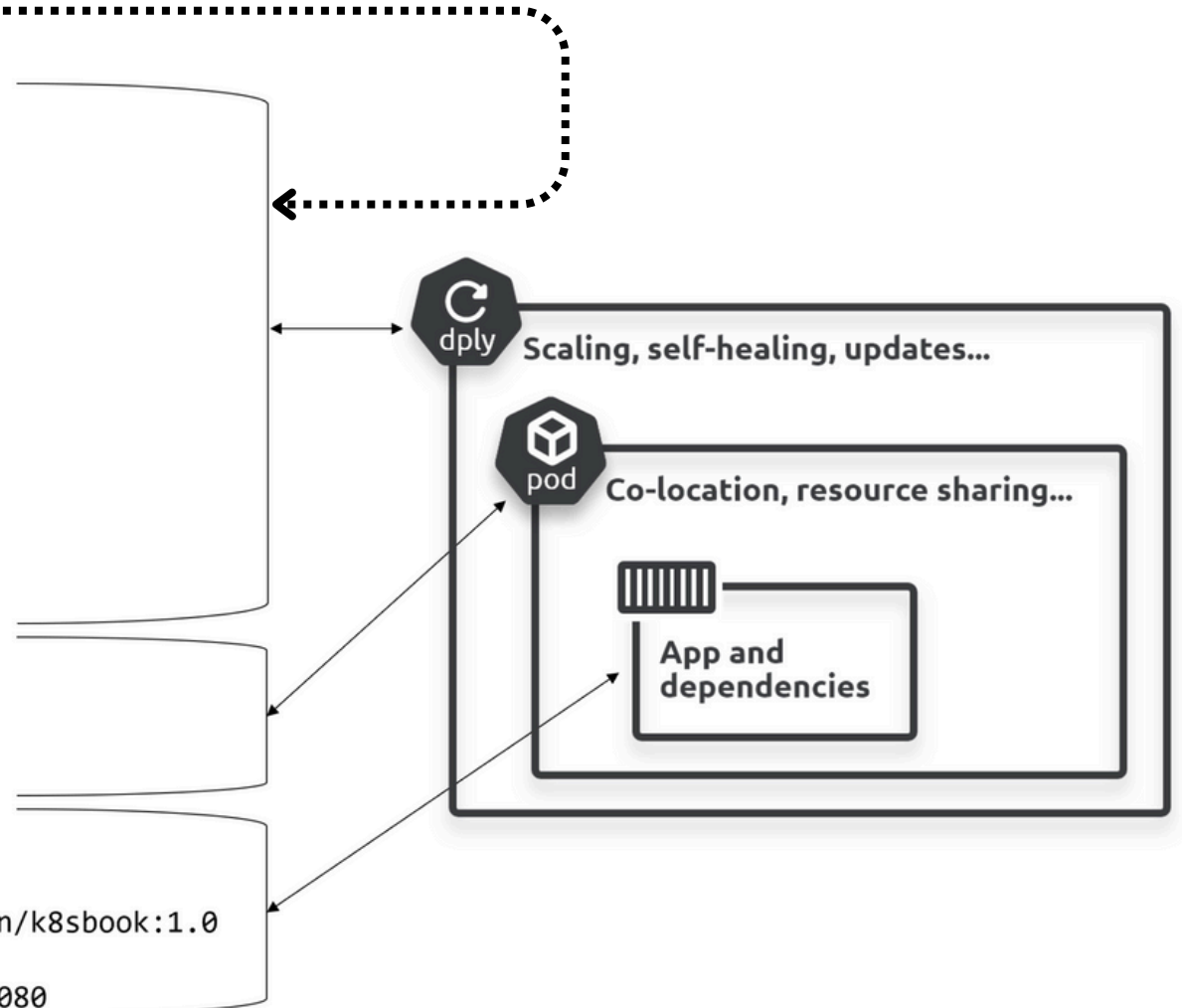
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 10
  selector:
    matchLabels:
      app: hello-world
  minReadySeconds: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-pod
          image: nigelpoulton/k8sbook:1.0
          ports:
            - containerPort: 8080
```



“Deployment” comme solution aux Pods

3. Enfin le Deployment enrichit encore davantage le tout avec des fonctionnalités de mise à l'échelle et de mise à jour.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 10
  selector:
    matchLabels:
      app: hello-world
  minReadySeconds: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-pod
          image: nigelpoulton/k8sbook:1.0
          ports:
            - containerPort: 8080
```



“Deployment” et pods

- Un objet **Deployment** ne gère qu'un seul modèle de **Pod**. Par exemple, une application avec un service web front-end et un catalogue en back-end aura un **Pod** différent pour chacun (deux modèles de **Pod**).
- Par conséquent, elle aura besoin de deux objets **Deployment** : l'un gérant les **Pods** front-end, l'autre gérant les **Pods** back-end.

Note: un objet **Deployment** peut gérer plusieurs répliques du même **Pod**.

Deployment et ReplicaSet

ReplicaSet : un autre objet sur lequel les **Deployment** s'appuient fortement en coulisse.

- Les **conteneurs** sont un excellent moyen de regrouper les applications et les dépendances.
- Les **Pods** permettent aux **conteneurs** de fonctionner sur Kubernetes et permettent la co-planification et toute une série d'autres avantages.

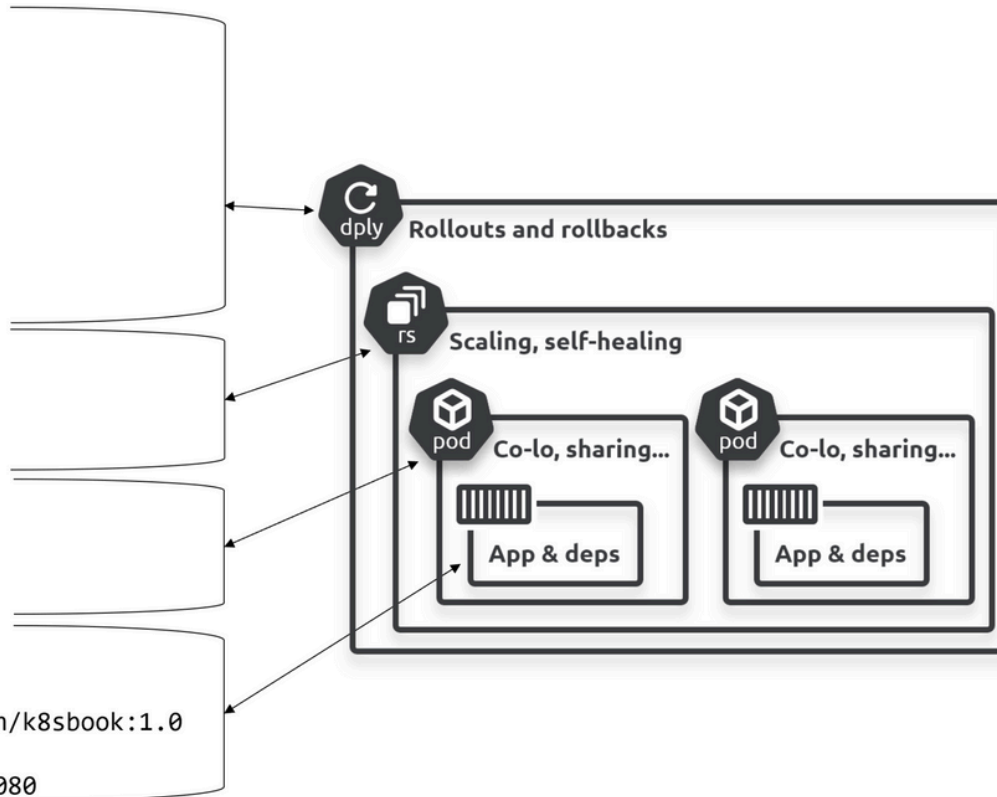
Deployment et ReplicaSet

ReplicaSet : un autre objet sur lequel les **Deployment** s'appuient fortement en coulisse.

- Les **ReplicaSets** gèrent les **Pods** et apportent **auto-réparation** et **mise à l'échelle**.
- Les **déploiements** gèrent les **ReplicaSets** et ajoutent les **déploiements** et les rollout/rollback.

Deployment et ReplicaSet

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  replicas: 10
  selector:
    matchLabels:
      app: hello-world
  minReadySeconds: 10
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-pod
          image: nigelpoulton/k8sbook:1.0
          ports:
            - containerPort: 8080
```



La figure est similaire à la figure précédente, mais ajoute un **ReplicaSet** à la relation et indique quel objet est responsable de quelles fonctionnalités.

Considérez les **déploiements** comme gérant les **ReplicaSets**, et les **ReplicaSets** comme gérant les **pods**.

Self-healing and scalability

Les pods sont formidables.

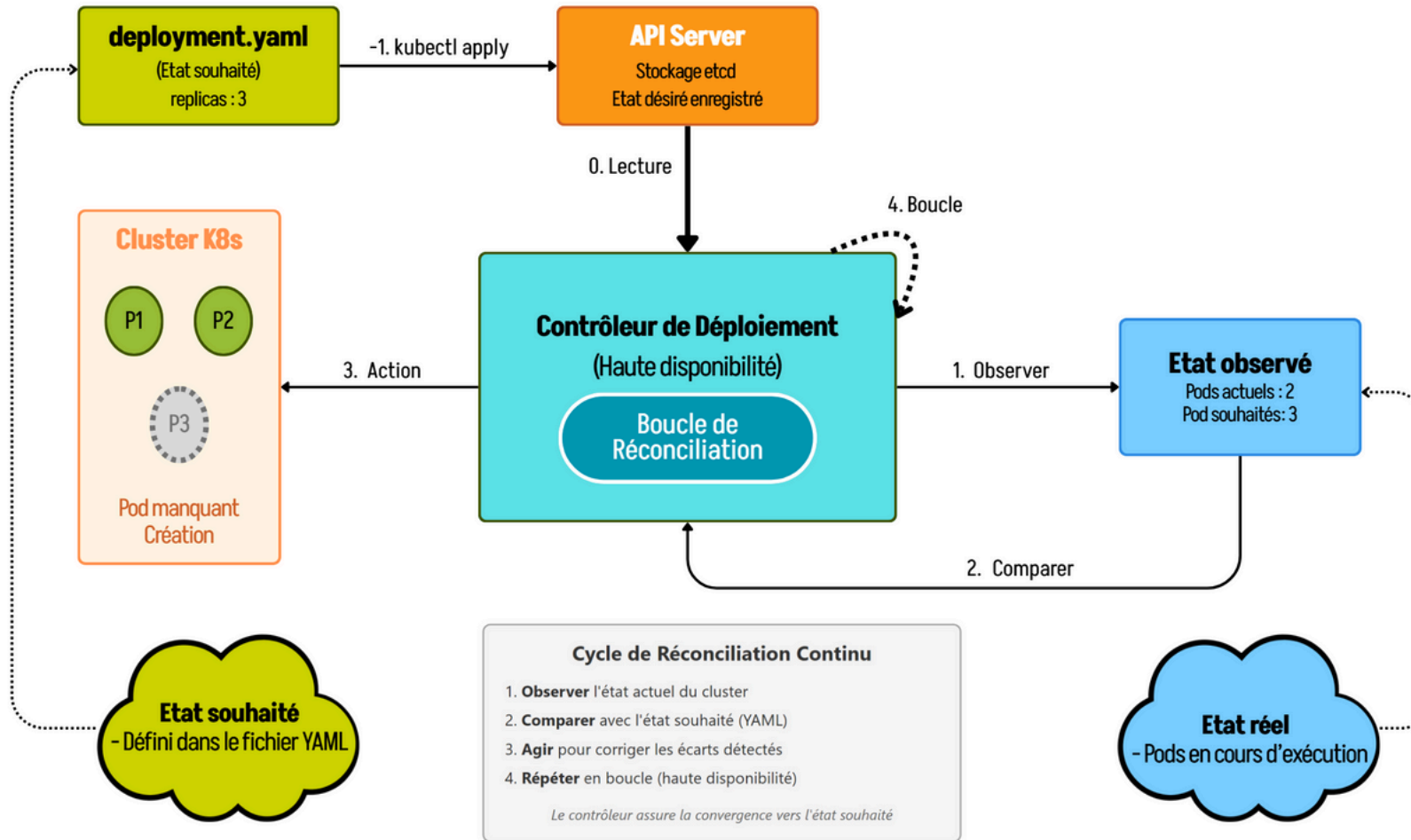
Ils vous permettent de :

- regrouper des conteneurs,
- partager des volumes,
- partager de la mémoire,
- simplifier la mise en réseau,
- et bien plus encore.
- Cependant ...

Les pods sont limités !

Ils n'offrent rien en matière d'**auto-réparation** et d'**évolutivité** : si le nœud sur lequel un **pod** est exécuté tombe en panne, le **pod** est perdu !!!

Boucle de réconciliation

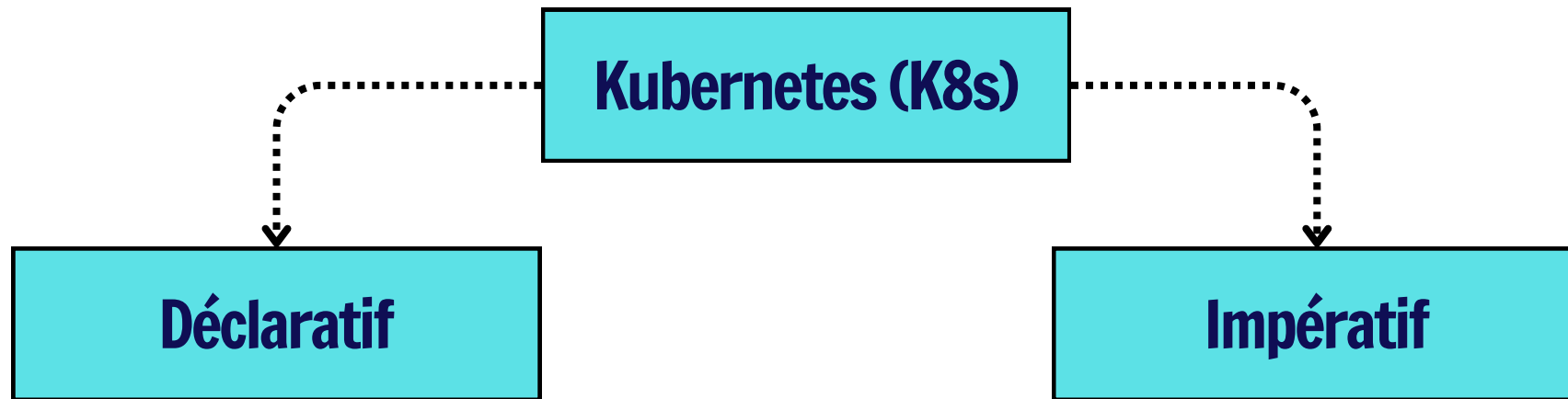


General rule of reconciliation

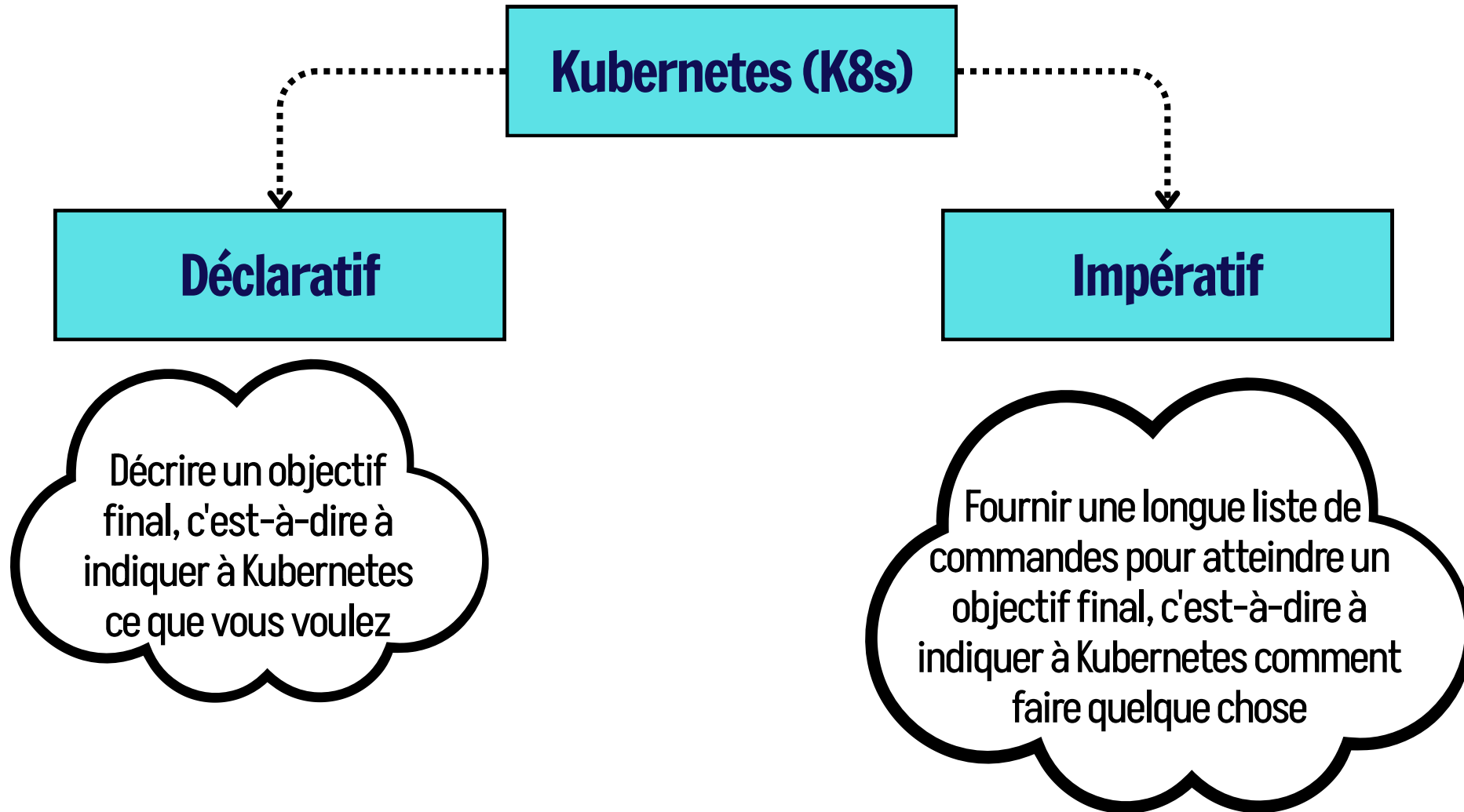
If **Desired state** match **Observed state**, everybody's happy.

If they don't match, a process of reconciliation brings them back together.

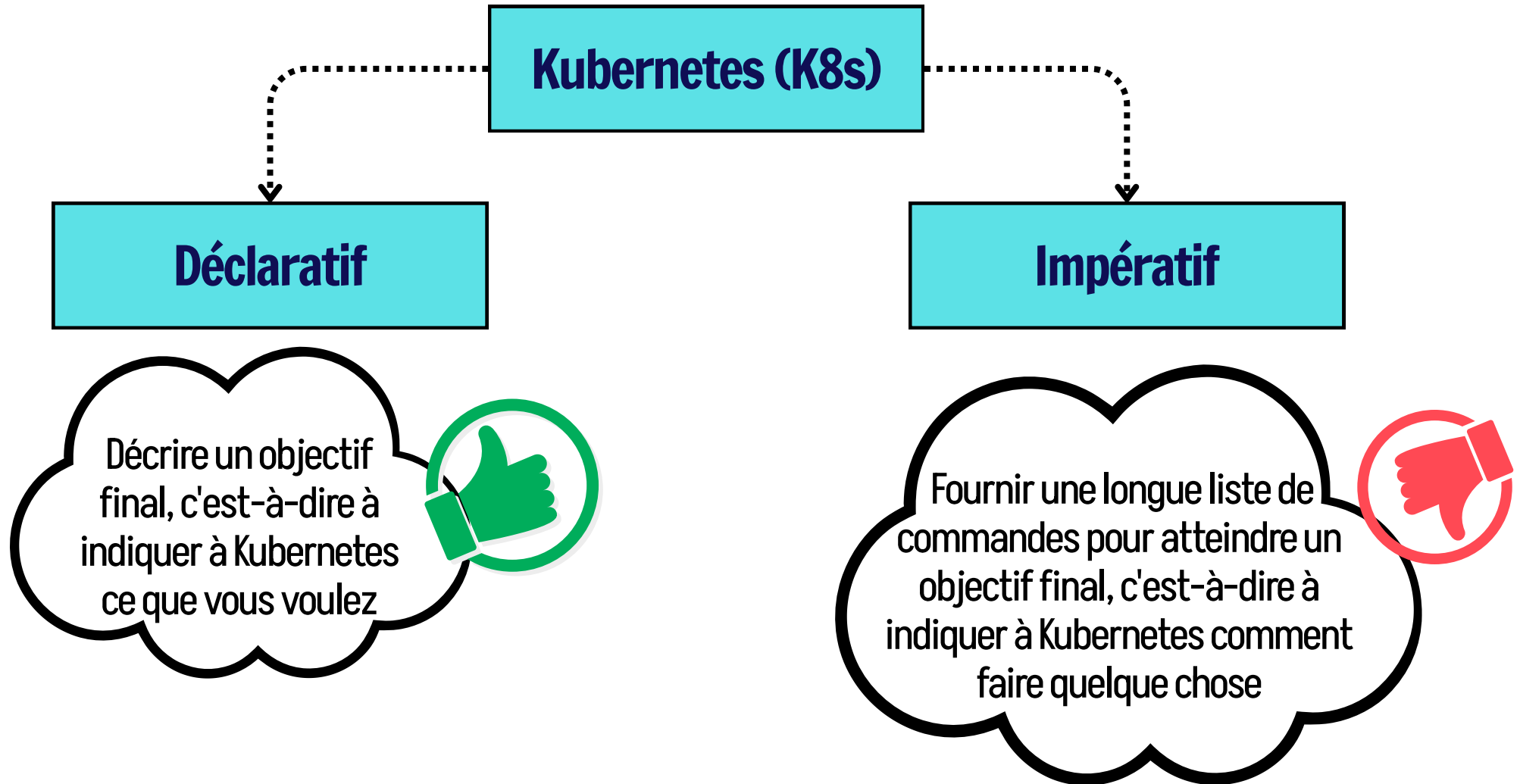
Modèle déclaratif vs Modèle affirmatif



Modèle déclaratif vs Modèle affirmatif



Modèle déclaratif vs Modèle affirmatif

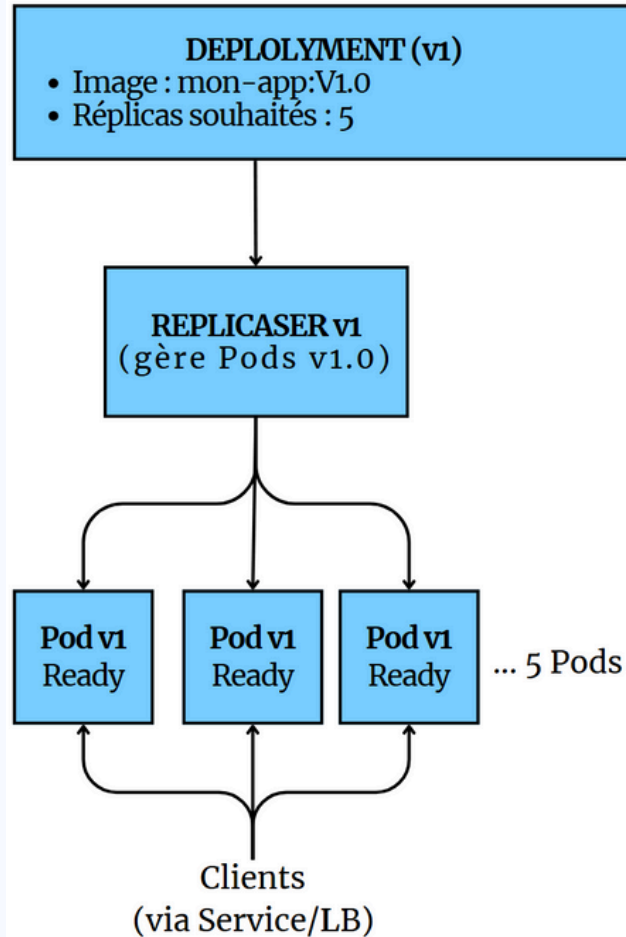


Rolling updates avec les déploiements

- Les **mise à jour continues**, sans interruption de service des applications sans état, sont l'essence même des **déploiements**.
- Cependant, elles nécessitent **certaines conditions** de la part de vos applications microservices pour fonctionner correctement.
 1. **Couplage via des API (microservices or serverless)**
 2. **Compatibilité ascendante et descendante**

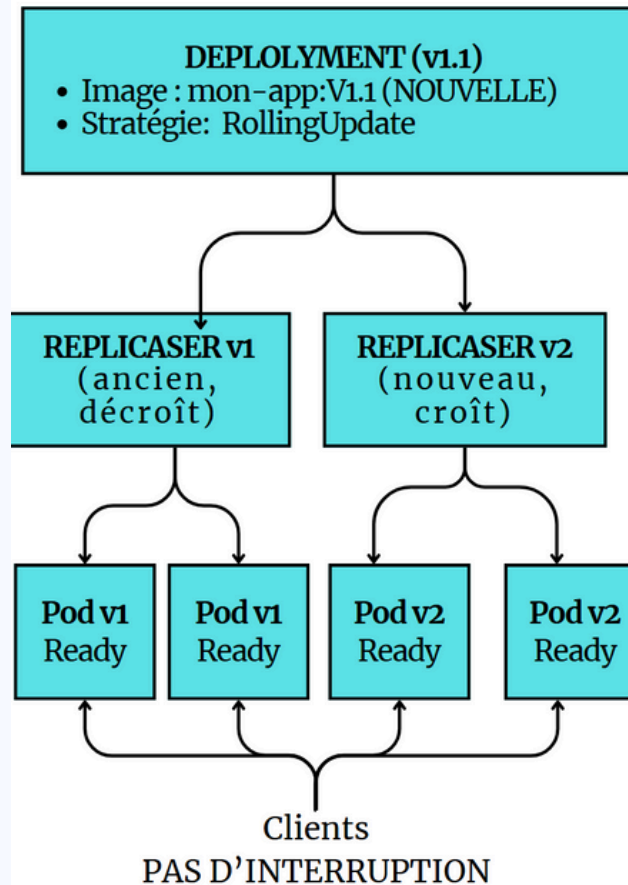
Rolling updates avec les déploiements

ETAT INITIALE

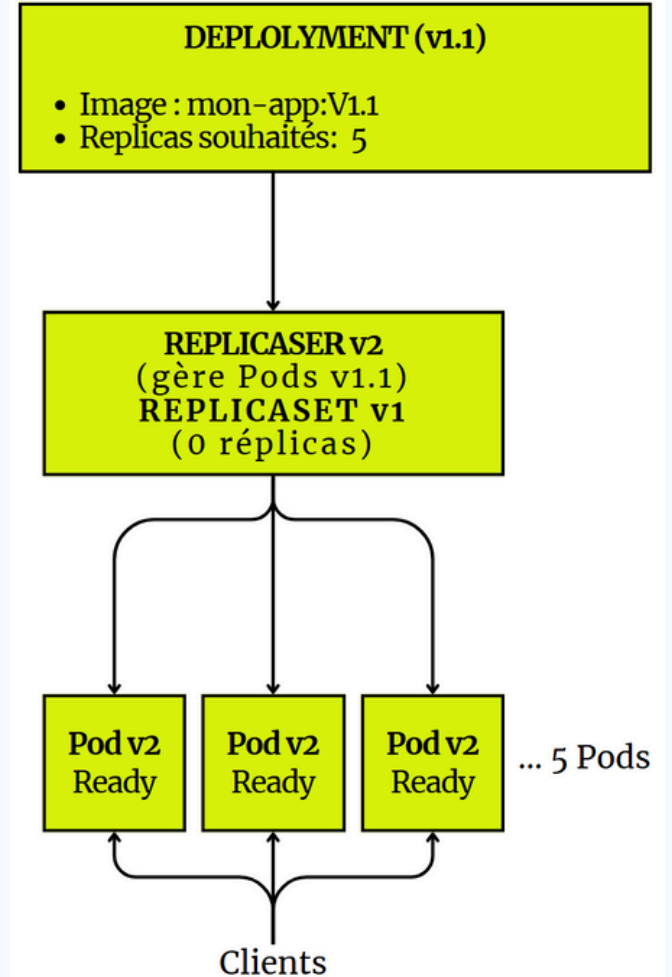


Mise à jour déclenché (kubectl apply v1.1)

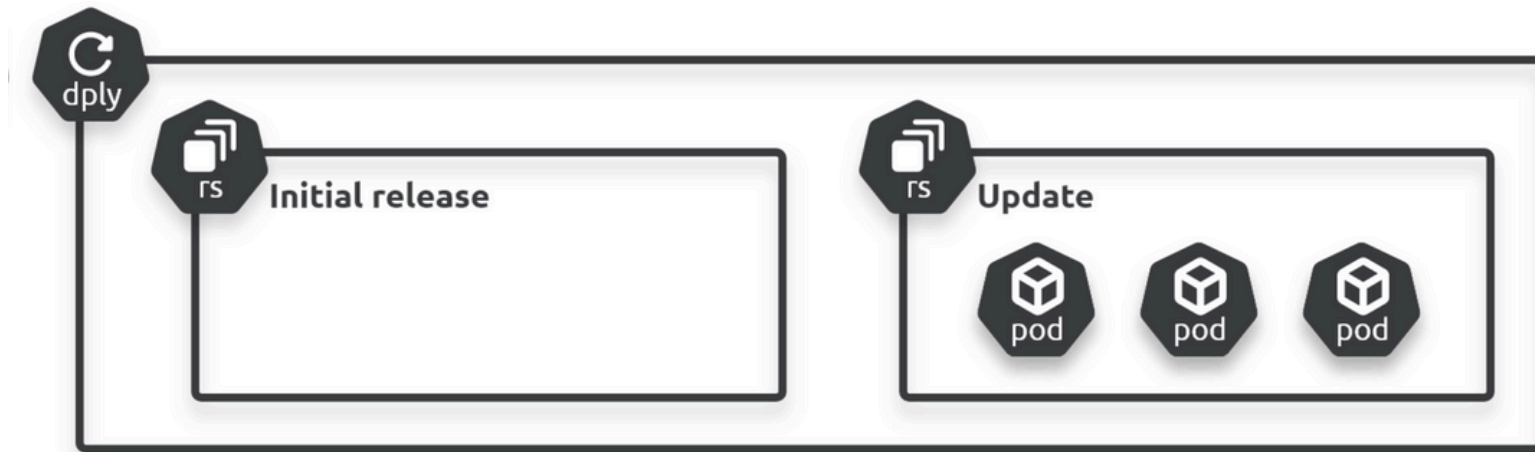
PHASE DE TRANSITION



ETAT FINAL



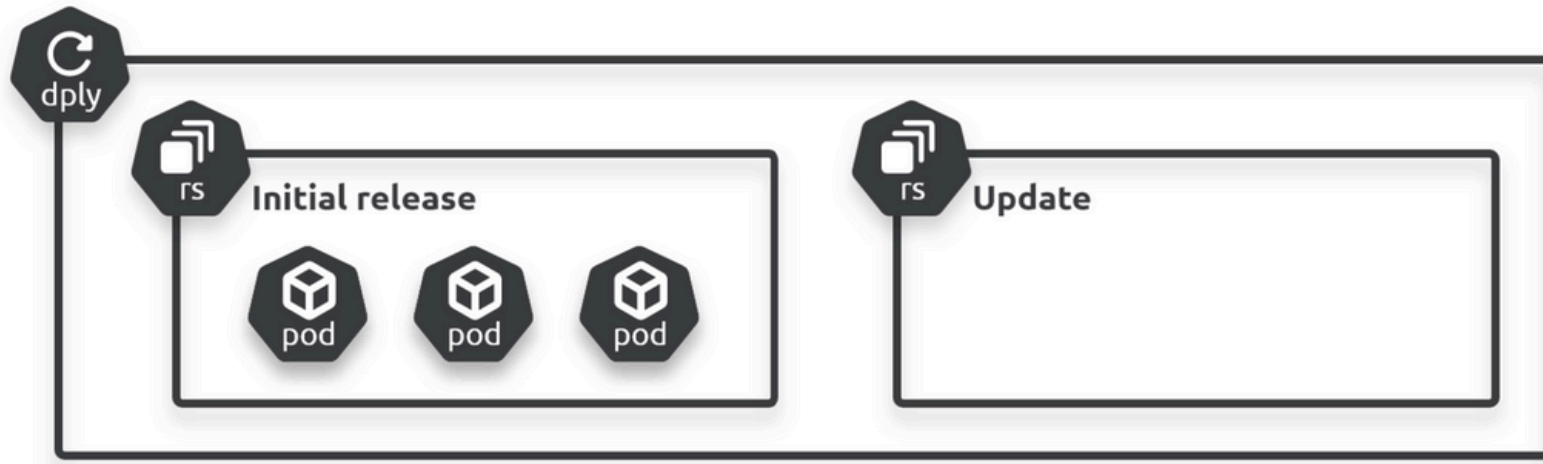
Rollout vs Rollback



Le schéma précédent peut aussi être représenté par celui-ci après l'étape finale de la mise à jour de l'app (v1.0 vers v1.1)

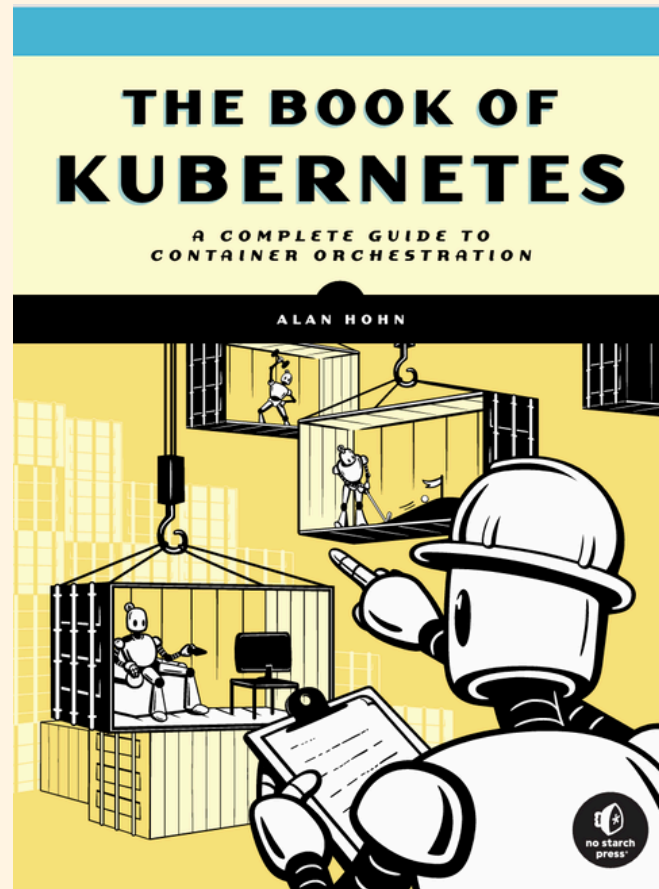
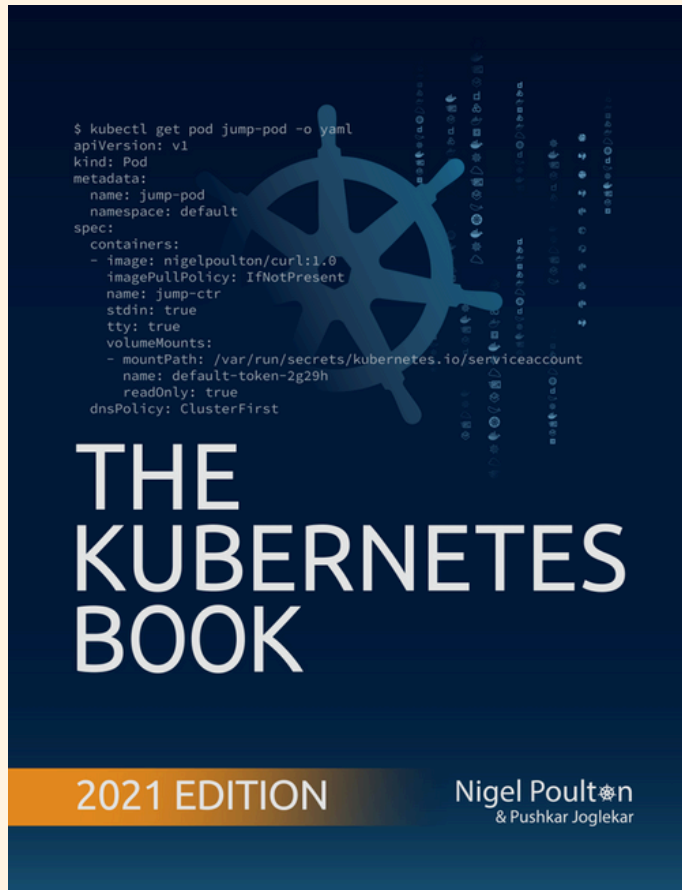
Il est important que l'ancien ReplicaSet de la version initiale existe toujours avec sa configuration intacte.

Rollout vs Rollback



Le processus de **rollback** est l'inverse d'un **rollout** : vous remontez l'un des anciens **ReplicaSets** tout en descendant celui qui est actuellement utilisé.
La figure ci-dessus montre la même application revenue à sa version initiale.

Documentation



Kubernetes Official Website Documentation

Kubernetes Documentation

Kubernetes is an open source container orchestration engine for automating deployment, scaling, and management of containerized applications. The open source project is hosted by the Cloud Native...

